

ESCUELA
POLITÉCNICA
NACIONAL

ESCUELA POLITÉCNICA NACIONAL

FACULTAD DE INGENIERÍA DE SISTEMAS

INGENIERÍA DE SOFTWARE

**VERIFICACIÓN Y VALIDACIÓN DE
SOFTWARE**

Nombre: Juan Diego Suárez Barberán

Curso: GR1SW

Fecha: 21/11/2025

1. CONTENIDO

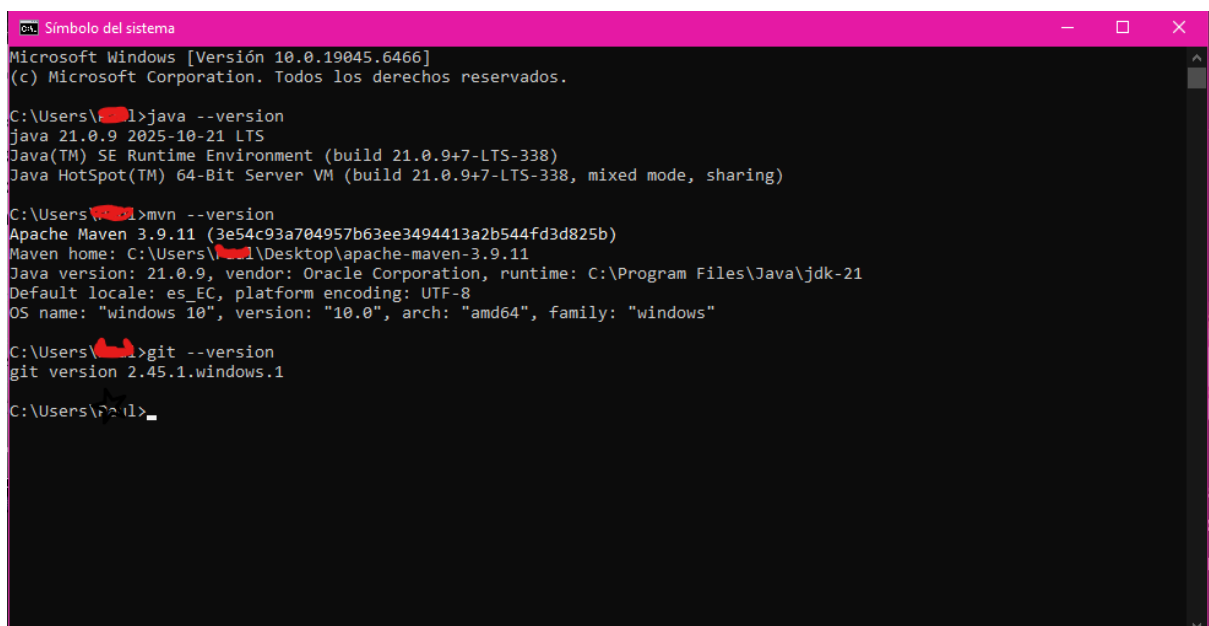
En el presente informe se describe el laboratorio de pruebas unitarias realizado sobre un proyecto desarrollado en Java [1]. El código fuente del proyecto se encuentra disponible en un repositorio público en GitHub para su revisión y validación [2].

1.1 Objetivo del laboratorio

El objetivo del laboratorio es aplicar los fundamentos de las pruebas unitarias en Java mediante el uso del framework JUnit. Para ello, se clonará un proyecto existente desde GitHub disponible en: <https://github.com/christian-suarezepn/junit-ex1-blueprint>. Luego, se implementarán casos de prueba siguiendo la metodología Arrange–Act–Assert (AAA), con el fin de validar el correcto funcionamiento de las operaciones básicas de la calculadora.

1.2 Desarrollo

Así, se comprueba que las herramientas necesarias para realizar el laboratorio estén configuradas y listas para trabajar:



```
Simbolo del sistema
Microsoft Windows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\...>java --version
java 21.0.9 2025-10-21 LTS
Java(TM) SE Runtime Environment (build 21.0.9+7-LTS-338)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.9+7-LTS-338, mixed mode, sharing)

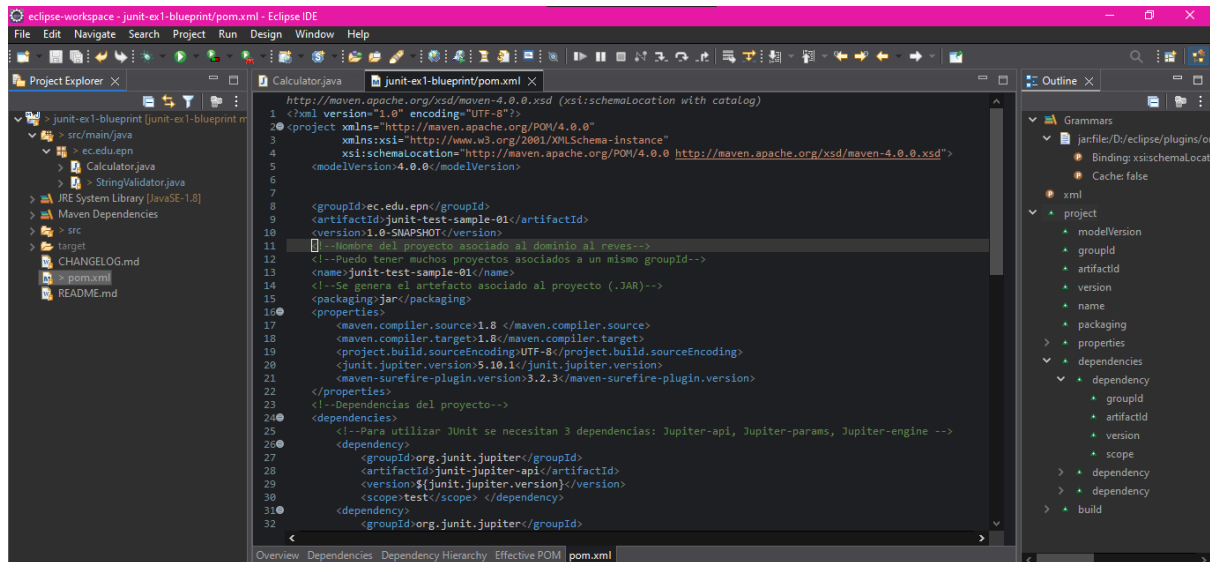
C:\Users\...>mvn --version
Apache Maven 3.9.11 (3e54c93a704957b63ee3494413a2b544fd3d825b)
Maven home: C:\Users\...\Desktop\apache-maven-3.9.11
Java version: 21.0.9, vendor: Oracle Corporation, runtime: C:\Program Files\Java\jdk-21
Default locale: es_EC, platform encoding: UTF-8
OS name: "windows 10", version: "10.0", arch: "amd64", family: "windows"

C:\Users\...>git --version
git version 2.45.1.windows.1

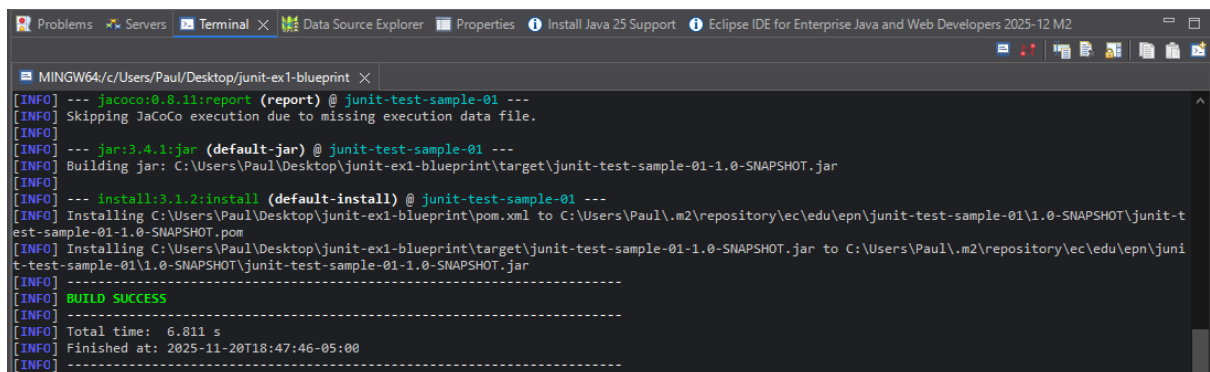
C:\Users\...>
```

- **Java 21.0.9**
- **Apache Maven 3.9.11**
- **Git 2.45.1**
- **Eclipse IDE for Web Developers**

Luego, se pasó a explicar las propiedades del archivo POM.XML del proyecto. Así mismo, el funcionamiento de Maven al momento de realizar los comandos para instalar las dependencias necesarias para JUnit.



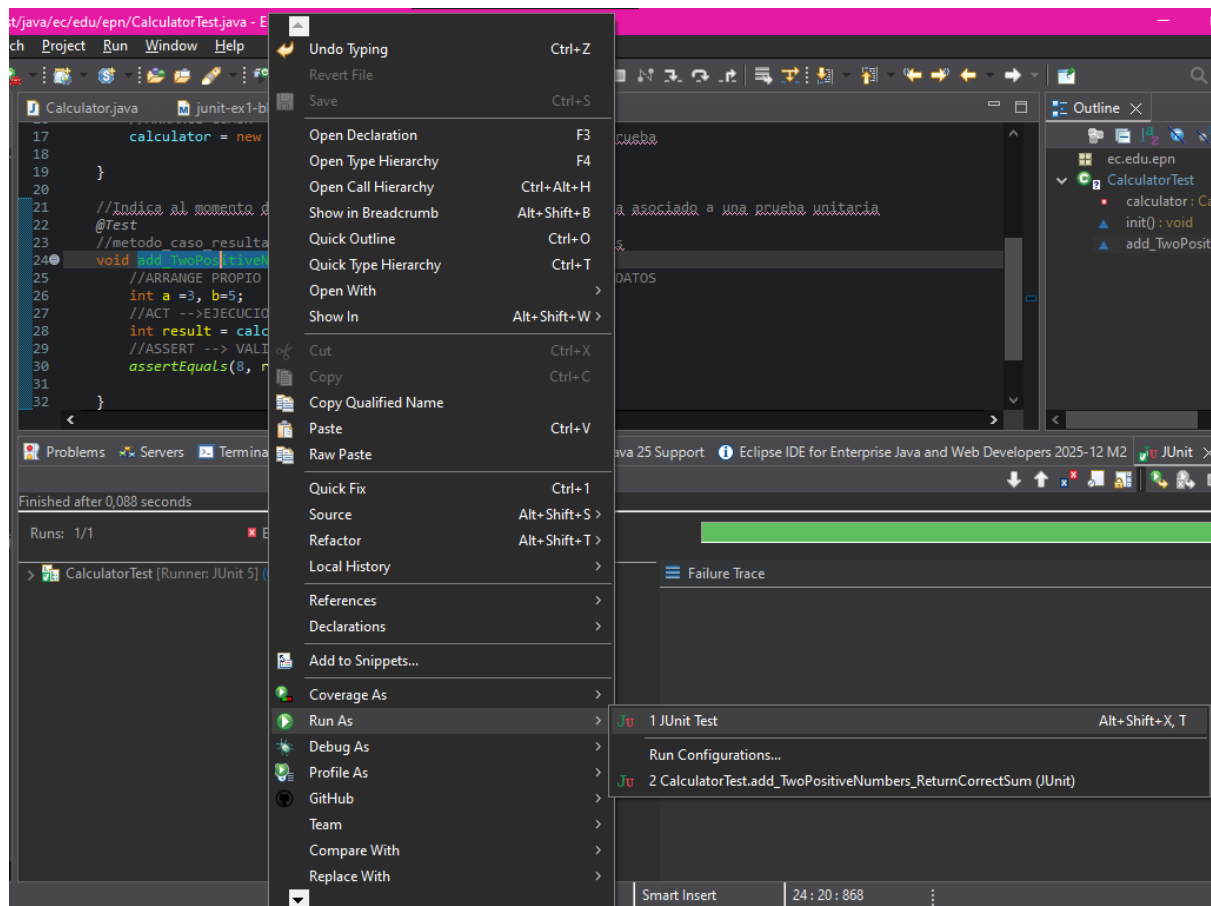
A continuación, se ejecutó en el bash integrado en el IDE el comando “mvn clean install” con el objetivo de bajar las dependencias (junit-api, junit-params y junit-engine). Por lo tanto, se puede observar como se instala en la ruta `\\.m2\\repository`



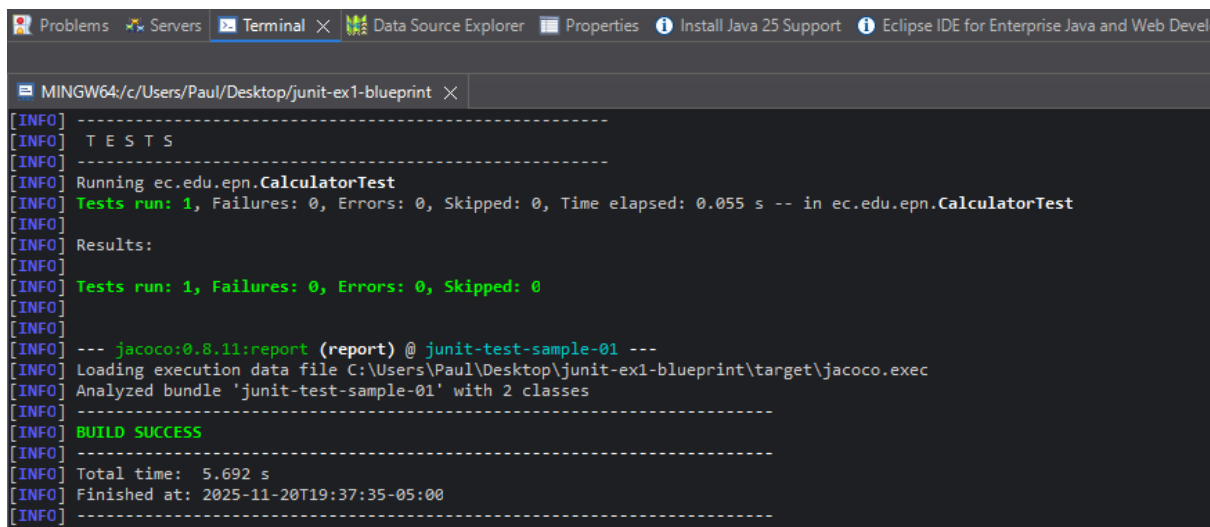
Pruebas de integración

Se configuró la estructura del proyecto para empezar con las pruebas de integración, agregando “test-java-CalculatorTest”. Es un estándar, nombrar a las clases de prueba con el mismo nombre de la clase a ser probada. Así mismo, existe un formato

METODO-CASO-RESULTADO_ESPERADO para detallar el nombre de una prueba unitaria, cómo se observa en la ilustración la ejecución del método `add_twoPositive_numbers` en la cuál estamos aplicando la metodología AAA (Arrange, Act, Assert). Finalmente, vemos el resultado de la prueba y como ha “pasado” con éxito.

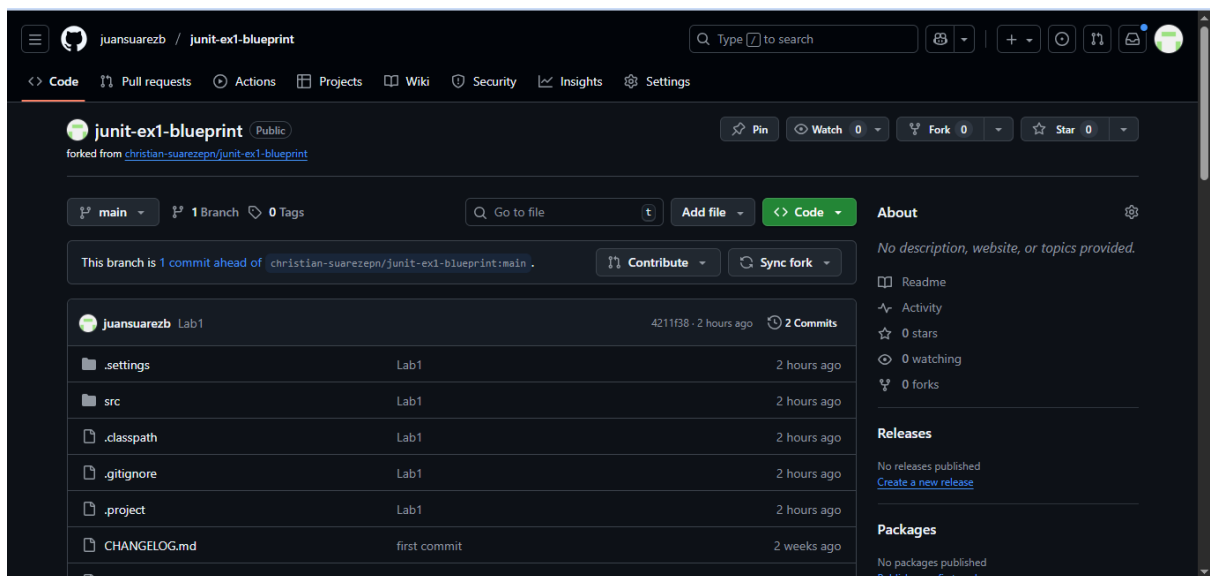


Luego, se comprobó a través de la misma terminal integrada del IDE Eclipse que se pueden ejecutar las pruebas a través del comando `mvn test`. Así, todos los métodos que tengan la anotación de Java `@Test` serán probadas y obtendremos los resultados en la misma terminal.



```
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running ec.edu.epn.CalculatorTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.055 s -- in ec.edu.epn.CalculatorTest
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jacoco:0.8.11:report (report) @ junit-test-sample-01 ---
[INFO] Loading execution data file C:\Users\Paul\Desktop\junit-ex1-blueprint\target\jacoco.exec
[INFO] Analyzed bundle 'junit-test-sample-01' with 2 classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 5.692 s
[INFO] Finished at: 2025-11-20T19:37:35-05:00
[INFO] -----
```

Finalmente, se realizó un push al fork que tenemos en el repositorio de github (fork del proyecto mencionado al inicio). El fork se encuentra disponible en: <https://github.com/juansuarezb/junit-ex1-blueprint>



1. CONCLUSIONES

- La realización de pruebas unitarias en el proyecto permitió comprobar el funcionamiento de cada una de las operaciones de la calculadora.
- Aplicar la metodología Arrange–Act–Assert (AAA) permitió estructurar los casos de prueba de manera clara y consistente
- Constituyen un componente esencial en el desarrollo profesional de software, contribuyendo a sistemas más confiables, robustos y fáciles de evolucionar.

2. BIBLIOGRAFÍA

[1] K. Arnold, J. Gosling, and D. Holmes, *The Java Programming Language*, 4th ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2005.

[2] GitHub, “Repositorio del proyecto de pruebas unitarias,” GitHub. Disponible en: *[coloca aquí el enlace de tu repositorio]*. Accedido: 20-nov-2025.